

Лабораторная работа №11

Модель системы массового обслуживания M|M|1

Ланцова Яна Игоревна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Реализация модели системы массового обслуживания $M M 1$ в CPN Tools	7
3.2	Мониторинг параметров моделируемой системы	10
4	Выводы	16

Список иллюстраций

3.1	Декларации модели СМО	8
3.2	Декларации модели СМО	8
3.3	Граф и параметры сети системы обработки заявок в очереди	9
3.4	Граф и параметры генератора заявок системы	10
3.5	Граф и параметры процесса обработки заявок на сервере системы	10
3.6	Функция Predicate монитора Ostanovka и функция Observer монитора Queue Delay	11
3.7	log файл Queue_Delay	12
3.8	График изменения задержки в очереди	13
3.9	Функция Observer монитора Queue Delay Real	13
3.10	Содержимое Queue_Delay_Real.log	14
3.11	Функция Observer монитора Long Delay Time	15
3.12	Периоды времени, когда значения задержки в очереди превышали заданное значение	15

Список таблиц

1 Цель работы

Реализовать в CPN Tools модель системы массового обслуживания $M|M|1$.

2 Задание

- Реализовать в CPN Tools модель системы массового обслуживания $M|M|1$.
- Настроить мониторинг параметров моделируемой системы и нарисовать графики очереди.

3 Выполнение лабораторной работы

3.1 Реализация модели системы массового обслуживания M|M|1 в CPN Tools

Зададим декларации системы(рис. 3.1).

- фишки типа UNIT определяют моменты времени;
- фишки типа INT определяют моменты поступления заявок в систему.
- фишки типа JobType определяют 2 типа заявок — А и В;
- кортеж Job имеет 2 поля: jobType определяет тип работы, соответственно имеет тип JobType, поле AT имеет тип INT и используется для хранения времени нахождения заявки в системе;
- фишки Jobs — список заявок;
- фишки типа ServerxJob — определяют состояние сервера, занятого обработкой заявок. Переменные модели:
- proctime — определяет время обработки заявки;
- job — определяет тип заявки;
- jobs — определяет поступление заявок в очередь

```

▼ colset UNIT = int;
▼ System
  ▼ colset INT = int;
  ▼ colset UNIT = unit timed;
  ▼ colset Server = with server timed;
  ▼ colset JobType = with A | B;
  ▼ colset Job = record jobType : JobType *
    AT : INT;
  ▼ colset Jobs = list Job;
  ▼ colset ServerxJob = product Server * Job timed;
  ▼ var proctime : INT;
  ▼ var job : Job;
  ▼ var jobs : Jobs;

```

Рис. 3.1: Декларации модели СМО

Также зададим функции (рис. 3.2) `expTime` (описывает генерацию целочисленных значений через интервалы времени, распределённые по экспоненциальному закону), `intTime` (преобразует текущее модельное время в целое число), `newJob` (возвращает значение из набора `Job` — случайный выбор типа заявки).

```

▼ fun expTime (mean: int) =
  let
    val realMean = Real.fromInt mean
    val rv = exponential((1.0/realMean))
  in
    floor (rv+0.5)
  end;
▼ fun intTime() = IntInf.toInt (time());
▼ fun newJob() = {jobType = JobType.ran(),
  AT = intTime()}

```

Рис. 3.2: Декларации модели СМО

Модель состоит из трех отдельных листов: на первом листе опишем граф системы (рис. 3.3), на втором — генератор заявок (рис. 3.4); на третьем — сервер обработки заявок (рис. 3.5).

Сеть имеет 2 позиции (очередь — `Queue`, обслуженные заявки — `Completed`) и два перехода (генерировать заявку — `Arrivals`, передать заявку на обработку серверу — `Server`). Переходы имеют сложную иерархическую структуру, задаваемую на

отдельных листах модели (с помощью соответствующего инструмента меню - Hierarchy).

Между переходом Arrivals и позицией Queue, а также между позицией Queue и переходом Server установлена дуплексная связь. Между переходом Server и позицией Completed — односторонняя связь.

Граф генератора заявок имеет 3 позиции (текущая заявка — Init, следующая заявка — Next, очередь — Queue из листа System) и 2 перехода (Init — определяет распределение поступления заявок по экспоненциальному закону с интенсивностью 100 заявок в единицу времени, Arrive — определяет поступление заявок в очередь).

Граф процесса обработки заявок на сервере имеет 4 позиции (Busy — сервер занят, Idle — сервер в режиме ожидания, Queue и Completed из листа System) и 2 перехода (Start — начать обработку заявки, Stop — закончить обработку заявки).

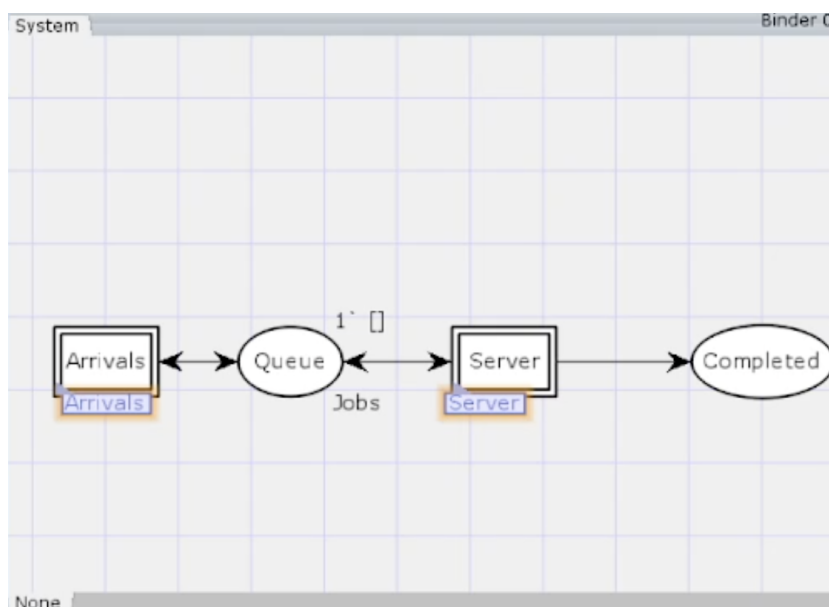


Рис. 3.3: Граф и параметры сети системы обработки заявок в очереди

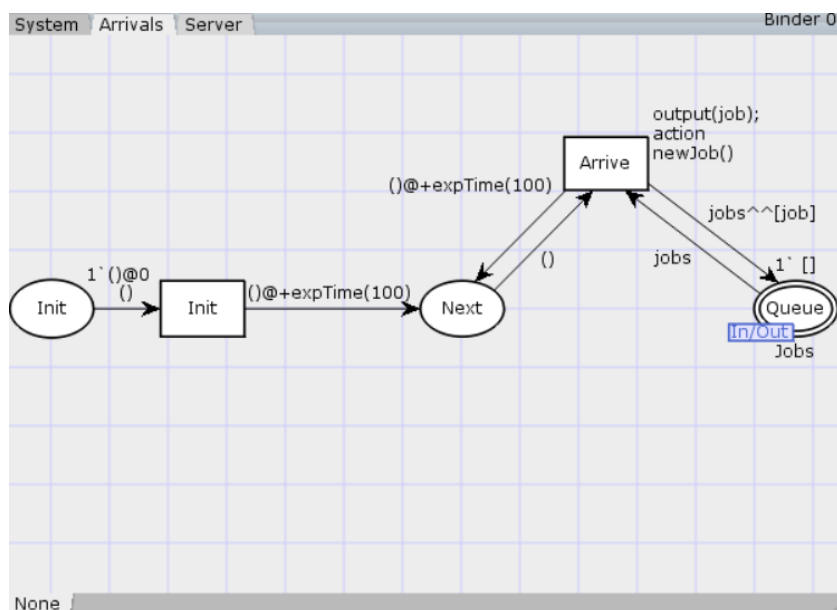


Рис. 3.4: Граф и параметры генератора заявок системы

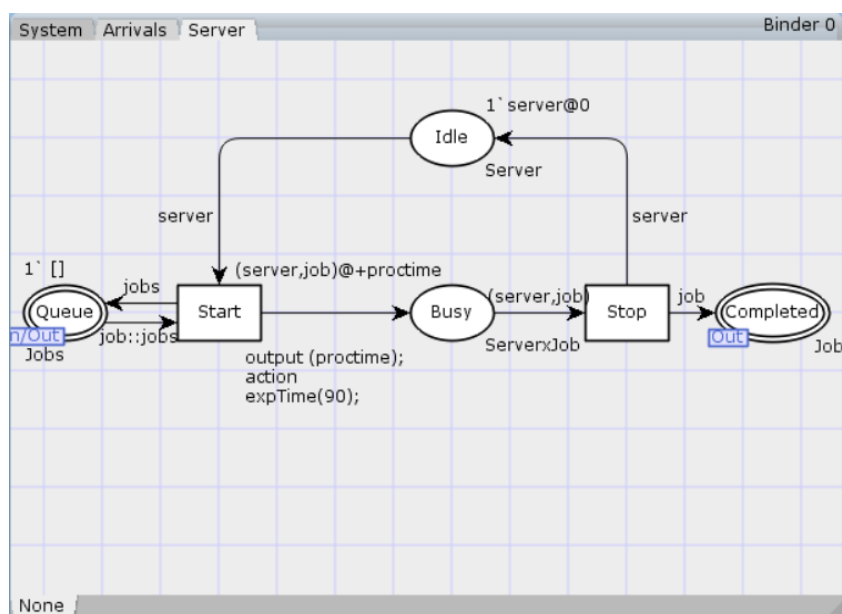


Рис. 3.5: Граф и параметры процесса обработки заявок на сервере системы

3.2 Мониторинг параметров моделируемой системы

Потребуется палитра Monitoring. Выбираем Break Point (точка останова) и устанавливаем её на переход Start. После этого в разделе меню Monitor появится

новый подраздел, который назовём Ostanovka. В этом подразделе необходимо внести изменения в функцию Predicate, которая будет выполняться при запуске монитора. Зададим число шагов, через которое будем останавливать мониторинг. Для этого true заменим на Queue_Delay.count()=200. Необходимо определить конструкцию Queue_Delay.count(). С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay (без подчеркивания). Функция Observer выполняется тогда, когда функция предикатора выдаёт значение true. По умолчанию функция выдаёт 0 или унарный минус (~1), подчёркивание обозначает произвольный аргумент. Изменим её так, чтобы получить значение задержки в очереди. Для этого необходимо из текущего времени intTime() вычесть временную метку AT, означающую приход заявки в очередь.

В результате функции примут вид(рис. 3.6). Из него может увидеть:

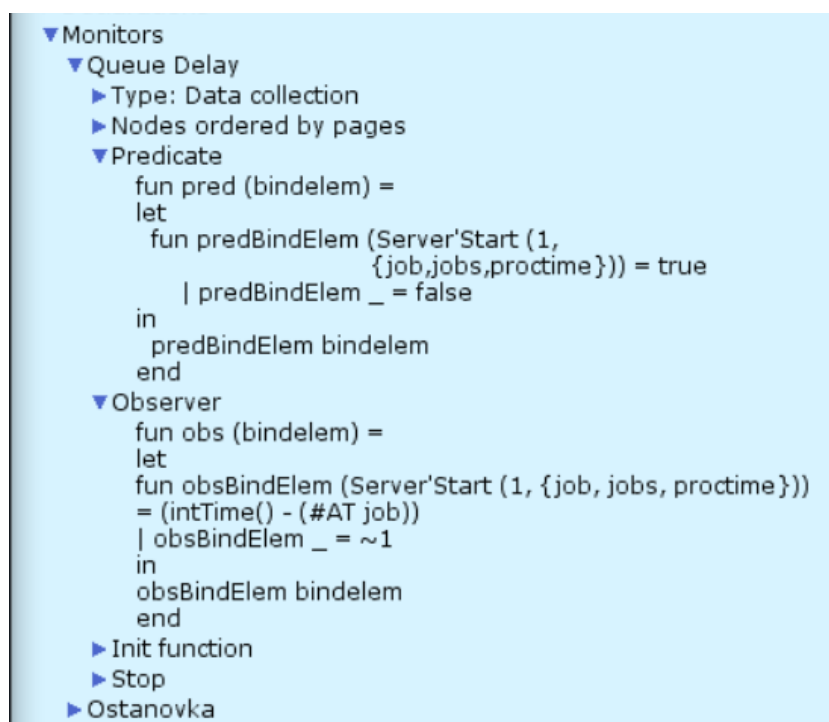
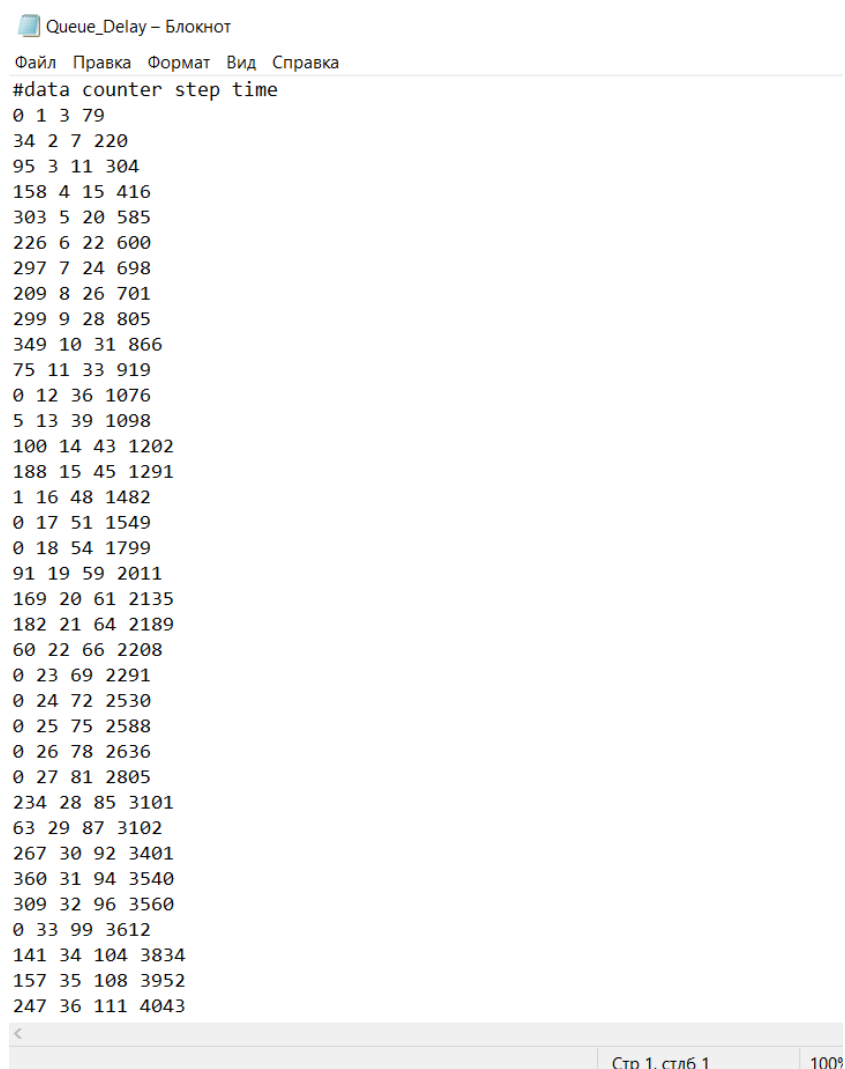


Рис. 3.6: Функция Predicate монитора Ostanovka и функция Observer монитора Queue Delay

После запуска программы на выполнение в каталоге с кодом программы по-

явится файл Queue_Delay.log (рис. 3.7), содержащий в первой колонке — значение задержки очереди, во второй — счётчик, в третьей — шаг, в четвёртой — время. С помощью gnuplot можно построить график значений задержки в очереди (рис. 3.8), выбрав по оси x время, а по оси y — значения задержки:



```

Queue_Delay - Блокнот
Файл  Правка  Формат  Вид  Справка
#data counter step time
0 1 3 79
34 2 7 220
95 3 11 304
158 4 15 416
303 5 20 585
226 6 22 600
297 7 24 698
209 8 26 701
299 9 28 805
349 10 31 866
75 11 33 919
0 12 36 1076
5 13 39 1098
100 14 43 1202
188 15 45 1291
1 16 48 1482
0 17 51 1549
0 18 54 1799
91 19 59 2011
169 20 61 2135
182 21 64 2189
60 22 66 2208
0 23 69 2291
0 24 72 2530
0 25 75 2588
0 26 78 2636
0 27 81 2805
234 28 85 3101
63 29 87 3102
267 30 92 3401
360 31 94 3540
309 32 96 3560
0 33 99 3612
141 34 104 3834
157 35 108 3952
247 36 111 4043

```

Рис. 3.7: log файл Queue_Delay

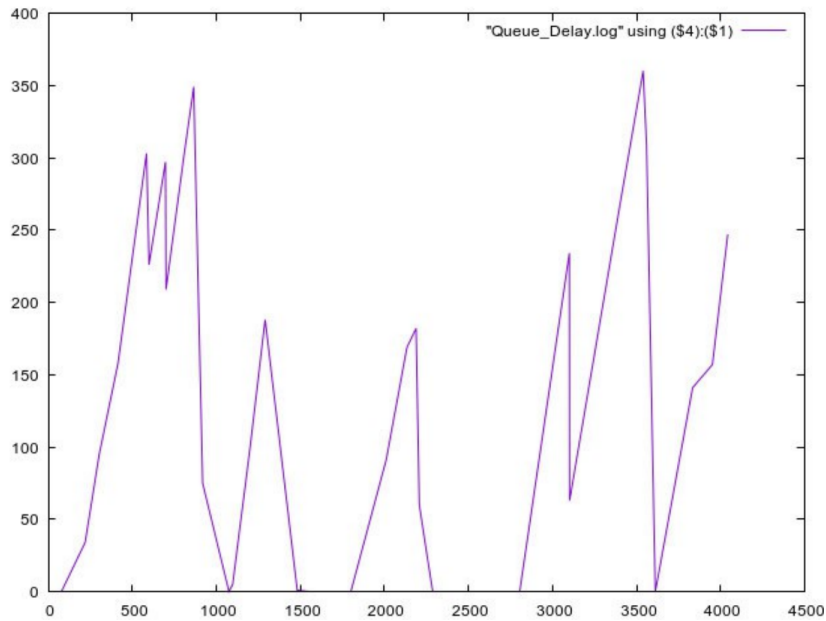


Рис. 3.8: График изменения задержки в очереди

Посчитаем задержку в действительных значениях. С помощью палитры Monitoring выбираем Data Call и устанавливаем на переходе Start. Появившийся в меню монитор называем Queue Delay Real. Функцию Observer изменим следующим образом(рис. 3.9):

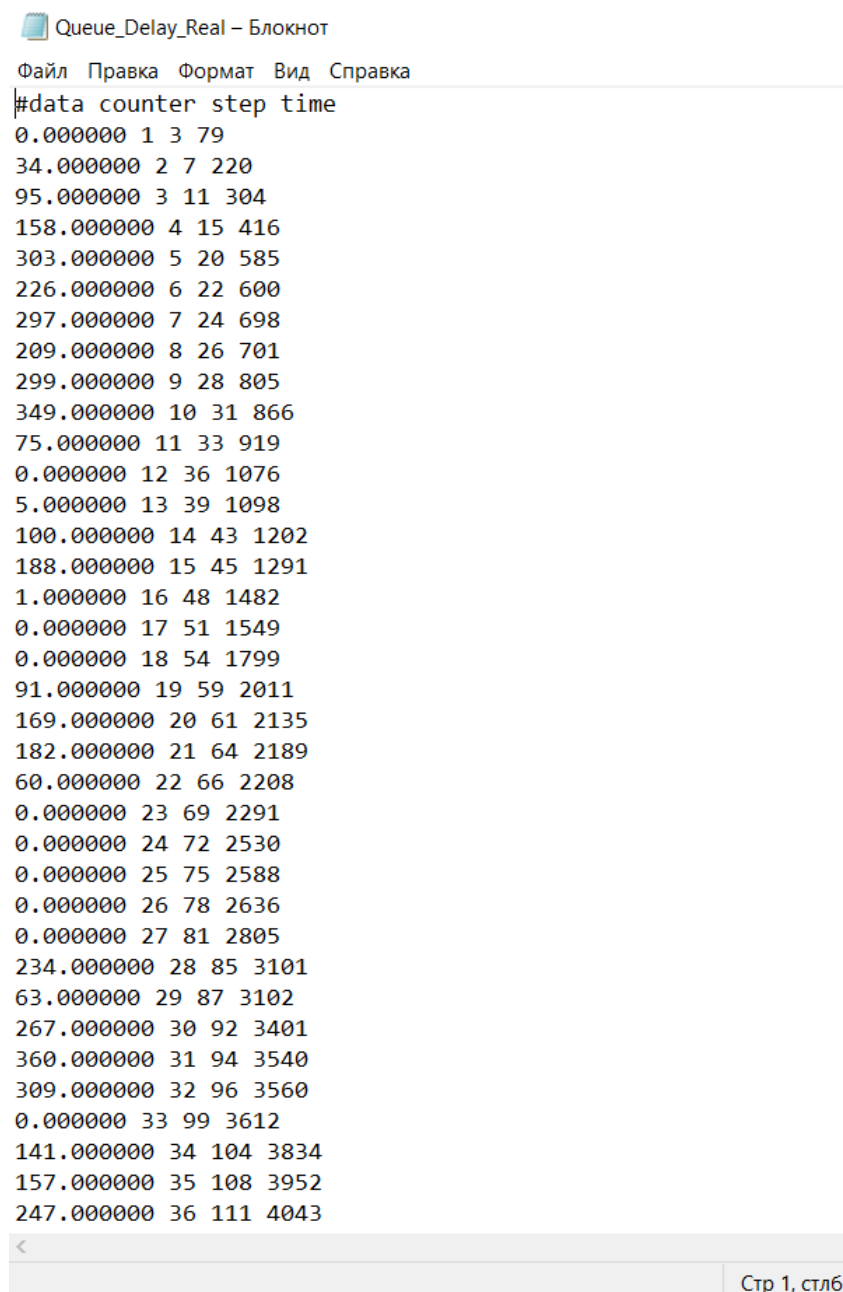
```

▼ Queue Delay Real
  ▶ Type: Data collection
  ▶ Nodes ordered by pages
  ▶ Predicate
  ▼ Observer
    fun obs (bindelem) =
      let
        fun obsBindElem (Server'Start (1, {job, jobs, proctime}))
          = Real.fromInt(intTime() - (#AT job))
          | obsBindElem _ = ~1.0
        in
          obsBindElem bindelem
        end
      in
        obsBindElem bindelem
      end
  ▶ Init function
  ▶ Stop
  
```

Рис. 3.9: Функция Observer монитора Queue Delay Real

По сравнению с предыдущим описанием функции добавлено преобразование значения функции из целого в действительное, при этом obsBindElem принима-

ет значение ~1.0. После запуска программы на выполнение в каталоге с кодом программы появится файл Queue_Delay_Real.log с содержимым, аналогичным содержимому файла Queue_Delay.log, но значения задержки имеют действительный тип(рис. 3.10):



```

Queue_Delay_Real - Блокнот
Файл  Правка  Формат  Вид  Справка
#data counter step time
0.000000 1 3 79
34.000000 2 7 220
95.000000 3 11 304
158.000000 4 15 416
303.000000 5 20 585
226.000000 6 22 600
297.000000 7 24 698
209.000000 8 26 701
299.000000 9 28 805
349.000000 10 31 866
75.000000 11 33 919
0.000000 12 36 1076
5.000000 13 39 1098
100.000000 14 43 1202
188.000000 15 45 1291
1.000000 16 48 1482
0.000000 17 51 1549
0.000000 18 54 1799
91.000000 19 59 2011
169.000000 20 61 2135
182.000000 21 64 2189
60.000000 22 66 2208
0.000000 23 69 2291
0.000000 24 72 2530
0.000000 25 75 2588
0.000000 26 78 2636
0.000000 27 81 2805
234.000000 28 85 3101
63.000000 29 87 3102
267.000000 30 92 3401
360.000000 31 94 3540
309.000000 32 96 3560
0.000000 33 99 3612
141.000000 34 104 3834
157.000000 35 108 3952
247.000000 36 111 4043
<
Стр 1, столб

```

Рис. 3.10: Содержимое Queue_Delay_Real.log

Посчитаем, сколько раз задержка превысила заданное значение. С помощью па-

литры Monitoring выбираем Data Call и устанавливаем на переходе Start. Монитор называем Long Delay Time. Функцию Observer изменим следующим образом(рис. 3.11):

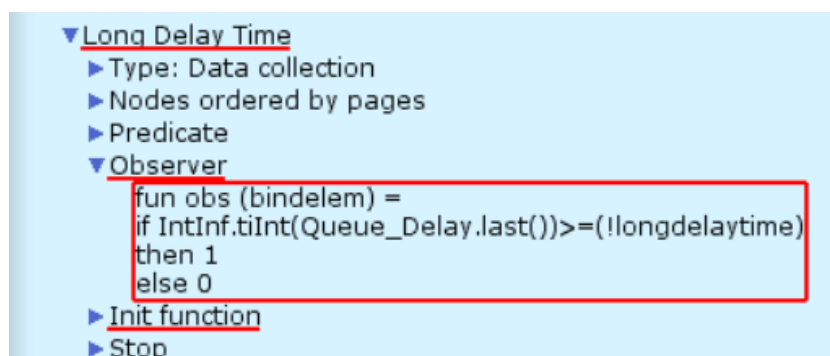


Рис. 3.11: Функция Observer монитора Long Delay Time

С помощью gnuplot можно построить график (рис. 3.12), демонстрирующий, в какие периоды времени значения задержки в очереди превышали заданное значение 200.

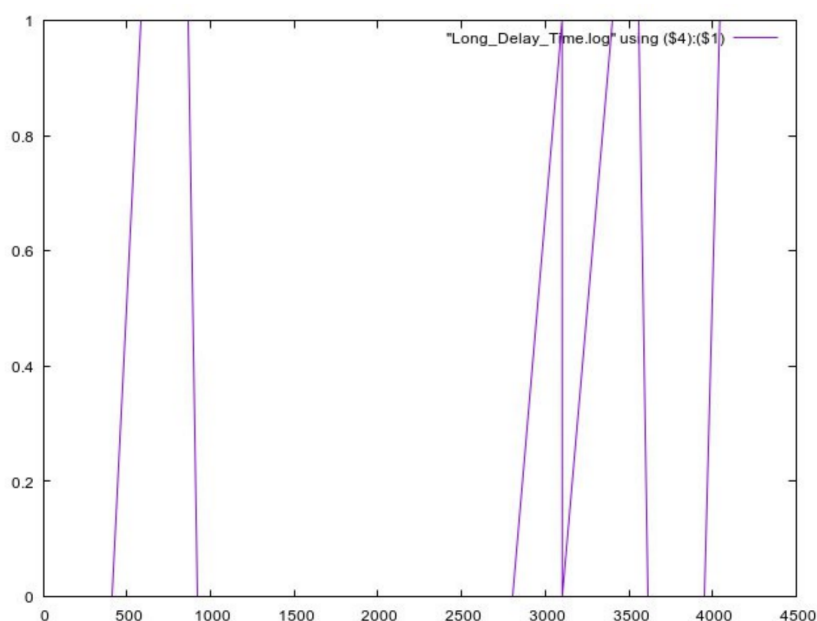


Рис. 3.12: Периоды времени, когда значения задержки в очереди превышали заданное значение

4 Выводы

В результате выполнения работы была реализована в CPN Tools модель системы массового обслуживания $M|M|1$.